Trabajo Práctico - Unidad 5

Calidad de Software: Pruebas Estáticas y Dinámicas (QA)

Cátedra: Metodología de Sistemas II - UTN TUP

Actividad: Trabajo en equipo - Entrega martes 27 de octubre

Proyecto: “Educar para Transformar” - Sistema de Gestión Escolar NovaSYS

Stack tecnológico: Node.js, Express, PostgreSQL (Neon Cloud), HTML5/Bootstrap 5, Vanilla JS (ES Modules).

Introducción

El presente trabajo aborda los contenidos de la Unidad 5 - Calidad de Software: pruebas estáticas y dinámicas - aplicados a una funcionalidad del proyecto NovaSYS distinta de la trabajada en la Unidad 4. Se selecciona un requerimiento funcional, se diseña un caso de prueba formal de verificación, se desarrolla una revisión estática mediante Peer Review, se documentan los entregables técnicos exigidos por la cátedra (historia de usuario, arquitectura, linter, pre-commit hooks, políticas de PR y orquestador centralizado), se implementa una prueba dinámica automatizada con Cypress y se definen dos métricas de calidad para evaluar el rendimiento del proceso.

Parte 2 (Trabajo en Equipo)

1. Funcionalidad seleccionada y requerimiento

Funcionalidad (diferente a la Unidad 4): Gestión, registro y consulta de fichas médicas de alumnos.

Requerimiento a verificar (RF-FM-01): *El sistema debe permitir al docente autenticado consultar, registrar y actualizar la ficha médica (grupo sanguíneo, alergias, tratamientos crónicos y contactos de urgencia) únicamente de los alumnos matriculados en los cursos y materias que tiene asignados, bloqueando el acceso a datos de alumnos de otros cursos por secreto y privacidad médica.*

2. Caso de prueba formal de verificación

Campo de caso de prueba	Descripción
ID de caso de prueba	CP-FM-01
Requerimiento	RF-FM-01: Registro, persistencia y control de acceso por rol/curso a la Ficha Médica de Alumnos.
Prueba diseñada por	Equipo de Control de Calidad (QA) – Metodología de Sistemas II
Fecha de diseño de la prueba	25 de octubre
Prueba ejecutada por	Tester de Equipo NovaSYS
Fecha de ejecución de la prueba	26 de octubre
Condición previa	1) Base de datos PostgreSQL conectada y con esquema de salud operativo (fichas_medicas). 2) Docente con usuario activo (profesor_primaria) y materias

Campo de caso de prueba	Descripción
	asignadas en 6° Grado Primaria. 3) Alumno A (id: 6, Joaquín Trangoni) inscripto en dicho curso (sin ficha inicial o con ficha editable). 4) Alumno B (Lucía García) inscripto en otro nivel/curso donde el docente NO dicta clases.

Detalle de pasos de prueba y resultados esperados:

N° Paso	Acción del paso	Resultado esperado
1	Iniciar sesión como Docente: ingresar a login.html, tipear correo docente y contraseña válida, presionar “Ingresar”.	Autenticación exitosa (HTTP 200). Se inicializa localStorage.usuario con rol docente y redirige a dashboard-docente.html.
2	Ingresar a la sección de fichas médicas: hacer clic en el acceso “Fichas Médicas / Salud Escolar” del menú lateral.	Se despliega el listado de alumnos. El sistema consulta a la API y muestra únicamente los estudiantes de los cursos asignados a este profesor.
3	Seleccionar un alumno, completar los datos y guardar: seleccionar a “Trangoni, Joaquín”, ingresar Grupo Sanguíneo: 0+, Alergias: Penicilina, Contacto Emergencia: 11-4567-8901 y hacer clic en “Guardar Ficha Médica”.	El botón muestra spinner de guardado. Se envía POST /api/fichas-medicas. La base de datos responde con inserción exitosa. Se muestra notificación Toast verde: “Ficha médica guardada correctamente en PostgreSQL”.
4	Reabrir la ficha del mismo alumno: cerrar el modal/vista, recargar la pantalla y volver a seleccionar al estudiante “Trangoni, Joaquín”.	Los campos se precargan automáticamente con los datos guardados en el paso 3 (0+, Penicilina, 11-4567-8901). Se valida la persistencia íntegra en la base de datos.
5	Buscar en el listado a un alumno de otra materia/curso: intentar buscar en el filtro por nombre/legajo a “García, Lucía” (estudiante de un curso no asignado a este docente).	El alumno no aparece en la lista de resultados. Si se intenta forzar la URL o llamada a la API con ese ID ajeno, el backend deniega la consulta respondiendo con HTTP 403 Forbidden (“No tiene permisos para ver la ficha médica de un alumno fuera de sus cursos”).

3. Prueba estática: Revisión por pares (Peer Review)

A. Estrategias de buenas prácticas para agilizar la evaluación del código

- Principio de Responsabilidad Única (SRP) y modularidad: separar controladores de rutas Express, consultas SQL a PostgreSQL y renderizado de interfaz en módulos independientes (fichasMedicasController.js, api.js, dashboard-docente.js), evitando archivos monolíticos de más de 300 líneas.
- Nombres semánticos y auto-documentados: variables y funciones con nombres descriptivos en español neutro (p. ej. obtenerFichaMedicaPorAlumno(), validarGrupoSanguineo()).
- Formateo estricto y automatizado: reglas de ESLint y Prettier obligatorias en el editor (tabulación de 2 espacios, comillas consistentes, punto y coma obligatorio) para que el revisor no pierda tiempo discutiendo estilo visual.

- Commits pequeños y semánticos: aplicar la convención Conventional Commits (feat:, fix:, test:, docs:) con descripciones precisas del cambio realizado.
- Pull Requests acotados: cada PR debe abordar una sola tarea o User Story, sin superar los 200-300 renglones de cambio, para mantener una alta atención cognitiva en la revisión.

B. Estrategias para aplicar la técnica de Peer Review

- Regla de Branch Protection en GitHub: bloquear la rama main; nadie puede hacer git push directo, toda integración requiere obligatoriamente un Pull Request.
- Aprobación mínima requerida: todo Pull Request debe tener como mínimo 1 aprobación (approval) de un compañero de equipo tras inspeccionar el código.
- Checklist formal de revisión de PR: ¿cumple con la Historia de Usuario y los criterios de aceptación?, ¿maneja errores con bloques try/catch y respuestas HTTP acordes (400, 403, 404, 500)?, ¿evita inyecciones SQL mediante consultas parametrizadas ($1, $2)?, ¿pasan los tests automatizados locales y de CI sin fallas?
- Revisión asíncrona con comentarios de línea: utilizar los comentarios interactivos de GitHub PR para señalar sugerencias de mejora puntuales de forma constructiva.

4. Entregables técnicos para el martes 27

A. Historia de Usuario (HU-FM01)

Título: HU-FM01: Registro y Visualización de Fichas Médicas de Alumnos por Curso.

COMO Docente a cargo de un curso o asignatura, QUIERO consultar y registrar los antecedentes de salud y fichas médicas de los alumnos de mis cursos, PARA actuar de manera rápida y preventiva ante urgencias médicas y adaptar actividades escolares a sus condiciones físicas.

Criterios de aceptación (Gherkin):

- Escenario 1 (Acceso a alumnos asignados): Dado que el docente inició sesión correctamente, Cuando ingresa al módulo de fichas médicas de su curso, Entonces puede visualizar la lista de sus alumnos y ver/editar su información de salud.
- Escenario 2 (Privacidad y restricción de datos médicos): Dado un estudiante que no cursa ninguna materia con el docente logueado, Cuando el docente busca en el listado o intenta acceder a su ID de ficha médica, Entonces el sistema oculta el registro y deniega la consulta con mensaje de acceso restringido.
- Escenario 3 (Persistencia de ficha): Dado un alumno seleccionado, Cuando el docente ingresa grupo sanguíneo, alergias y contacto de emergencia y presiona “Guardar”, Entonces los datos se persisten en la tabla fichas_medicas de PostgreSQL y quedan disponibles para futuras consultas.

B. Arquitectura de la aplicación

Patrón arquitectónico: Cliente-servidor desacoplado con capas lógicas (arquitectura de 3 capas).

- Capa de presentación (frontend web): HTML5 semántico, Bootstrap 5, CSS3 custom, JavaScript Vanilla moderno (ES Modules). Consume la API REST mediante fetch asíncrono con async/await.
- Capa de negocio y servicios (backend REST API): Node.js con Express.js (API_Server.js). Controla autenticación basada en sesión/roles (RBAC), enrutamiento, validación de esquemas y lógica de negocio escolar.

- Capa de datos y persistencia: PostgreSQL Serverless en la nube (Neon Cloud). Conexión optimizada por Connection Pool (pg.Pool), transacciones ACID (BEGIN, COMMIT, ROLLBACK) y claves foráneas en cascada.

C. Configuración de linter en el IDE (Visual Studio Code)

Archivo de configuración .eslintrc.json en la raíz del proyecto:

```
{
  "env": { "browser": true, "es2021": true, "node": true },
  "extends": "eslint:recommended",
  "parserOptions": { "ecmaVersion": "latest", "sourceType": "module" },
  "rules": {
    "no-unused-vars": "warn",
    "no-console": "off",
    "semi": ["error", "always"],
    "quotes": ["error", "single"],
    "eqeqeq": ["error", "always"]
  }
}
```

D. Configuración de Pre-commit Hooks con Husky

Implementación para bloquear commits locales si el código contiene errores de sintaxis o formato.

1. Instalación de herramientas:

```
npm install --save-dev husky lint-staged
npx husky init
```

2. Configuración en package.json:

```
"lint-staged": {
  "*.js": ["eslint --fix"]
}
```

3. Script en .husky/pre-commit:

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx lint-staged
node "src/Backend/test_docente_grading.js"
```

Efecto: si un desarrollador tiene un error de sintaxis o rompe una prueba, Git aborta el commit localmente impidiendo subir código roto.

E. Políticas de Pull Requests (PR)

Plantilla estándar institucional .github/pull_request_template.md:

```
## Descripción del Cambio
¿Qué funcionalidad o corrección introduce este Pull Request? (Ej: HU-FM01 Fichas
Médicas).

## Checklist del Desarrollador
- [ ] El código cumple con las reglas de ESLint sin errores.
- [ ] Se ejecutaron y pasaron todas las pruebas automatizadas.
- [ ] No se exponen credenciales de base de datos ni claves secretas.
- [ ] La interfaz responde de forma adaptable (responsive).
```

```
## Revisión por Pares (Peer Reviewer)
- [ ] Revisión de lógica de negocio y seguridad aprobada.
```

F. Orquestador centralizado (SonarCloud / SonarQube)

- Función en el proyecto: escaneo automático del repositorio ante cada Pull Request.
- Quality Gate definido: cobertura de tests > 75 %; 0 vulnerabilidades de seguridad (Security Rating: A); deuda técnica menor a 30 minutos por sprint.

5. Prueba dinámica (Quality Assurance - QA)

A. Herramienta seleccionada: Cypress

Justificación técnica: Cypress es el estándar de la industria para pruebas end-to-end (E2E) e integración en aplicaciones web modernas basadas en JavaScript. A diferencia de Selenium (que requiere drivers pesados y configuración compleja de WebDrivers de navegador), Cypress se ejecuta directamente en el mismo bucle de eventos del navegador, ofreciendo pruebas ultra-rápidas, recarga en tiempo real, snapshots visuales de cada paso y sin problemas de sincronización de timers.

B. Configuración de la herramienta en el proyecto

4. Instalación:

```
npm install --save-dev cypress
```

5. Configuración en cypress.config.js:

```
const { defineConfig } = require("cypress");

module.exports = defineConfig({
  e2e: {
    baseUrl: "http://localhost:3000",
    viewportWidth: 1280,
    viewportHeight: 720,
  },
});
```

C. Ejemplo de prueba de regresión automatizada en Cypress (cypress/e2e/ficha_medica.cy.js)

Este script simula las acciones de un usuario real verificando que la funcionalidad no sufra regresiones:

```
describe('Prueba de Regresión E2E: Ficha Médica y Control de Acceso Docente', () => {

  beforeEach(() => {
    cy.visit('/html/login.html');
    cy.get('#txtEmail').type('nicolas.trangoni@novasys.edu.ar');
    cy.get('#txtPassword').type('docente123');
    cy.get('#btnIngresar').click();
    cy.url().should('include', 'dashboard-docente.html');
  });

  it('Debe consultar, registrar y persistir la ficha médica de un alumno asignado', ()
=> {
    cy.get('[data-section="fichas-medicas"]').click();
    cy.get('#seccionFichasMedicas').should('be.visible');

    cy.contains('Trangoni, Joaquín').parents('tr').find('.btn-ver-ficha').click();
    cy.get('#modalFichaMedica').should('be.visible');
```

```
    cy.get('#selectGrupoSanguineo').select('0+');
    cy.get('#txtAlergias').clear().type('Penicilina y polen');
    cy.get('#txtContactoUrgencia').clear().type('11-4567-8901 (Madre)');
    cy.get('#btnGuardarFichaMedica').click();

    cy.get('.toast-feedback').should('contain', 'Ficha médica guardada
correctamente');

    cy.visit('/html/dashboard-docente.html');
    cy.get('[data-section="fichas-medicas"]').click();
    cy.contains('Trangoni, Joaquín').parents('tr').find('.btn-ver-ficha').click();
    cy.get('#selectGrupoSanguineo').should('have.value', '0+');
    cy.get('#txtAlergias').should('have.value', 'Penicilina y polen');
  });

  it('No debe mostrar alumnos pertenecientes a cursos no asignados al docente', () =>
{
    cy.get('[data-section="fichas-medicas"]').click();
    cy.get('#filtroBuscarAlumno').type('Lucía García');
    cy.get('#tablaFichasMedicas').should('not.contain', 'Lucía García');
  });
});
```

6. Métricas de calidad y rendimiento

Métrica seleccionada	Definición y propósito	Herramienta de medición	Cómo se evalúa su rendimiento en NovaSYS
Cobertura de Código (Code Coverage)	Mide el porcentaje de líneas, funciones y bifurcaciones (if/else) del código ejecutadas por las pruebas automatizadas.	Istanbul / NYC / SonarCloud	Se ejecuta npm test -- --coverage. Ningún Pull Request se aprueba si la cobertura global del backend y frontend desciende por debajo del 80 %.
Tasa de Defectos Escapados (Defect Escape Rate)	Porcentaje de bugs encontrados tardíamente por los usuarios finales en producción respecto al total de fallos detectados internamente en QA.	GitHub Issues (etiquetado bug:prod vs bug:qa)	Fórmula: (Bugs en Producción / Bugs Totales) × 100. Un valor inferior al 5 % indica que el proceso de pruebas dinámicas (Cypress) y estáticas (Peer Review) es altamente efectivo antes del despliegue.

Conclusiones

La incorporación de pruebas estáticas y dinámicas sobre el módulo de fichas médicas de NovaSYS permitió verificar, de manera formal y reproducible, que el control de acceso por curso y la persistencia de datos sensibles funcionan correctamente. El Peer Review, apoyado en branch protection y checklist de revisión, reduce la probabilidad de que código inseguro o sin pruebas llegue a la rama principal, mientras que la automatización con Cypress garantiza que cada cambio futuro se valide contra el mismo escenario de uso real sin intervención manual. Finalmente, el seguimiento de la cobertura de código y la tasa de defectos escapados ofrece al equipo una medida objetiva y continua de la calidad del software entregado, cerrando el ciclo entre desarrollo, pruebas y mejora continua.